

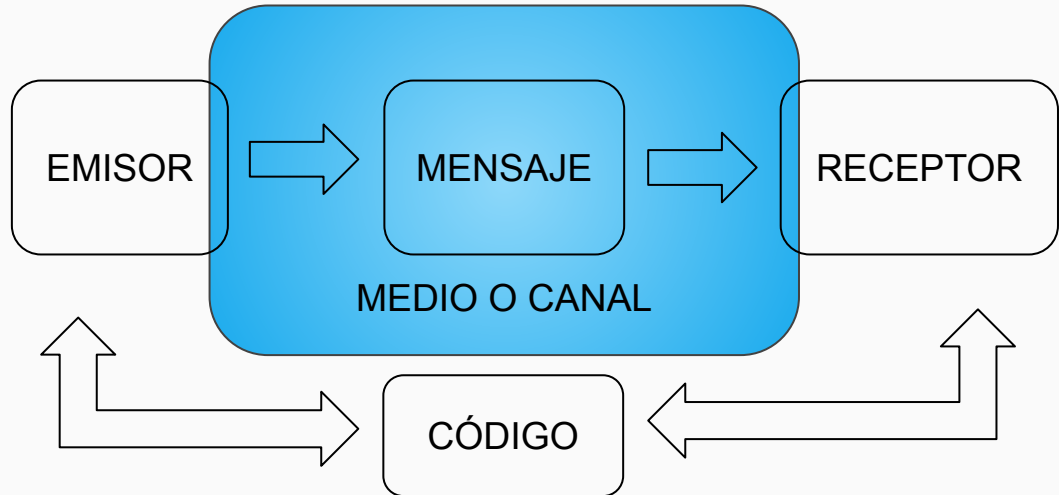
Códigos de detección de errores



Introducción

La comunicación está constituida por 5 elementos fundamentales:

1. EMISOR
2. RECEPTOR
3. MENSAJE
4. MEDIO o CANAL
5. CÓDIGO



Introducción

Cuando 2 personas hablan, una, cumple el rol de emisor, y produce o emite un mensaje.

En este ejemplo el mensaje podría ser un sonido, por ejemplo una palabra.

Este mensaje se desplaza por el aire que cumple la función de medio transmisor

y llega al receptor como movimientos de aire que éste decodifica en su oído y cobra sentido para él ya que comparten un código, en este caso el idioma.

Luego los roles se invierten y se vuelve a repetir el proceso.

Introducción

Esta definición también se usa en las comunicaciones digitales.

Por ejemplo en un paso de datos entre la RAM y la caché, la RAM es el emisor.

La caché es el receptor.

La cadena de bits es el mensaje.

El bus es el medio de comunicación.

Ambas memorias comparten el código del mensaje, no como programa sino como un standard (dirección de memoria, id de bloque, y datos con formato)

Introducción

El problema que suele aparecer generalmente en la comunicación es que el **medio** en el que se transporta el mensaje suele tener lo que se denomina **RUIDO**

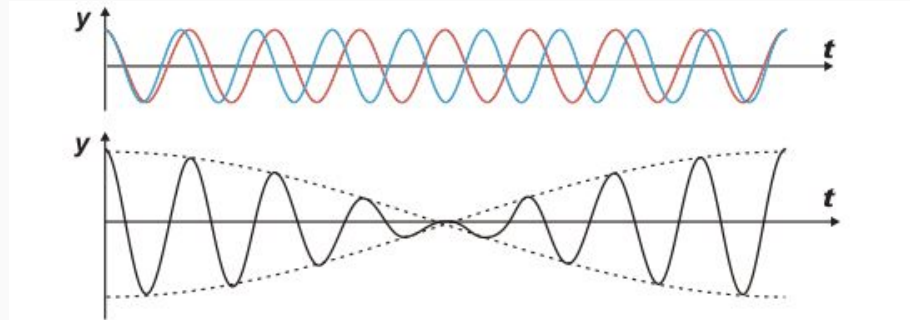
Volviendo al ejemplo de las personas charlando si hay más personas hablando o maquinaria funcionando, el mensaje puede **distorsionarse** dificultando la capacidad del receptor de poder **decodificar** el mensaje.

En las comunicaciones digitales esta distorsión también se la denomina **RUIDO**, pero las causas y los efectos son muy diferentes.

Causas de RUIDO o INTERFERENCIA

La transmisión y recepción de datos binarios desde un dispositivo a otro están propensos a errores por alteraciones externas del medio en el que se transmiten:

- campos magnéticos
- interferencias
- ruidos eléctricos
- caídas de tensión



Detección de errores

Cuando el mensaje sufre alguna de estas interferencias es importante que el sistema los detecte ya que un mensaje erróneo puede acarrear:

- un resultado erróneo (sistemas inconsistentes)
- búsqueda de bloques de memoria inexistentes (tiempos de ejecución enormes)
- pérdida de datos (falla en el objetivo del sistema)

Para evitar estos problemas los sistemas deben detectar y/o corregir errores de comunicación en el menor tiempo posible de manera que puedan mantener el intercambio de información digital en línea y en tiempo real.

Detección y Corrección de errores

¿Podemos saber si la información que llega es la misma que enviamos?

¿Puede el receptor reconocer si los datos que recibió son correctos?

La respuesta es sí a todo!! (yes to all)

Códigos de detección de errores: enviar información junto con los datos que permita deducir que un error ocurrió, pero no cual, y pida una retransmisión.

Códigos de corrección de errores: enviar información redundante junto con cada bloque de datos a enviar al receptor para deducir qué carácter se envió.

Definiciones

1. **Error**: un error en datos binarios es definido como un valor incorrecto en uno o más bits
2. **Single error**: valor incorrecto en un solo bit
3. **Multiple error**: uno o más bits incorrectos
4. **$d(I,J)$** : distancia entre I e J
número de posiciones de bits en los cuales las palabras I e J son diferentes
5. **$w(P)$** : peso de la palabra P
número de bits dentro de P iguales a 1

Ejemplos

Considerar las siguientes palabras

I = 01101100 J = 11000100

El peso de cada una es

$w(I) =$

$w(J) =$

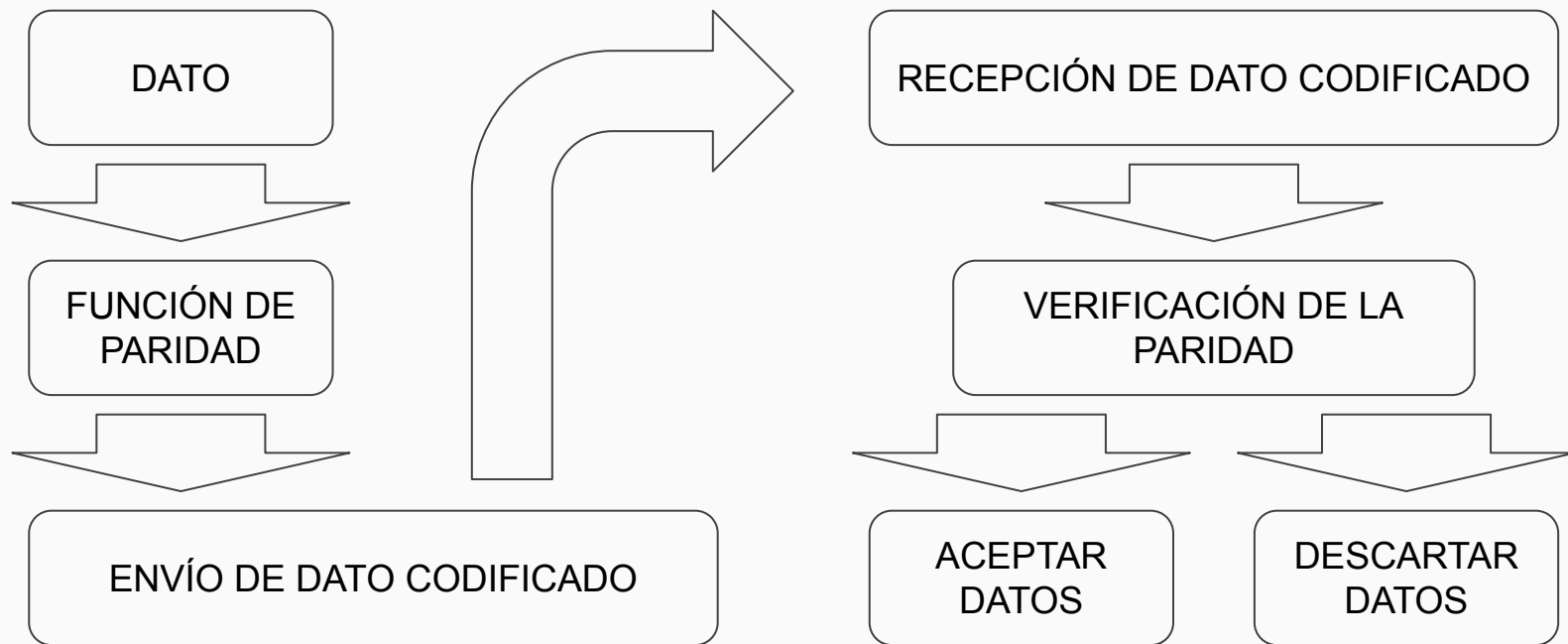
La distancia entre las dos es

$d(I, J) =$

Método por Paridad

- Este método consiste en establecer un tipo de **paridad** (par o impar) agregando un bit de modo que el peso del dato corresponda con la paridad.
Es decir que el peso corresponda a la paridad.
- Es muy utilizado en las comunicaciones seriales de datos.
- Este bit se agrega en la parte más significativa del dato (BMS).

Diagrama de funcionamiento



Ejemplos

Dados:

A) 1010

B) 1110101

C) 001001

Calcular las paridades:

Solución Par:

A) 01010

B) 11110101

C) 0001001

Solución Impar:

A) 11010

B) 01110101

C) 1001001

Conclusión

- Este método es eficiente en la detección de errores cuando hay confiabilidad.
- Solo se puede detectar errores de dos datos que difieran en un bit, sin embargo no los corrige y a lo sumo puede señalar el error y/o solicitar que vuelvan a enviar el dato, byte, palabra, bloque.

Método de detección por Hamming

Inventado por Richard Hamming en 1950

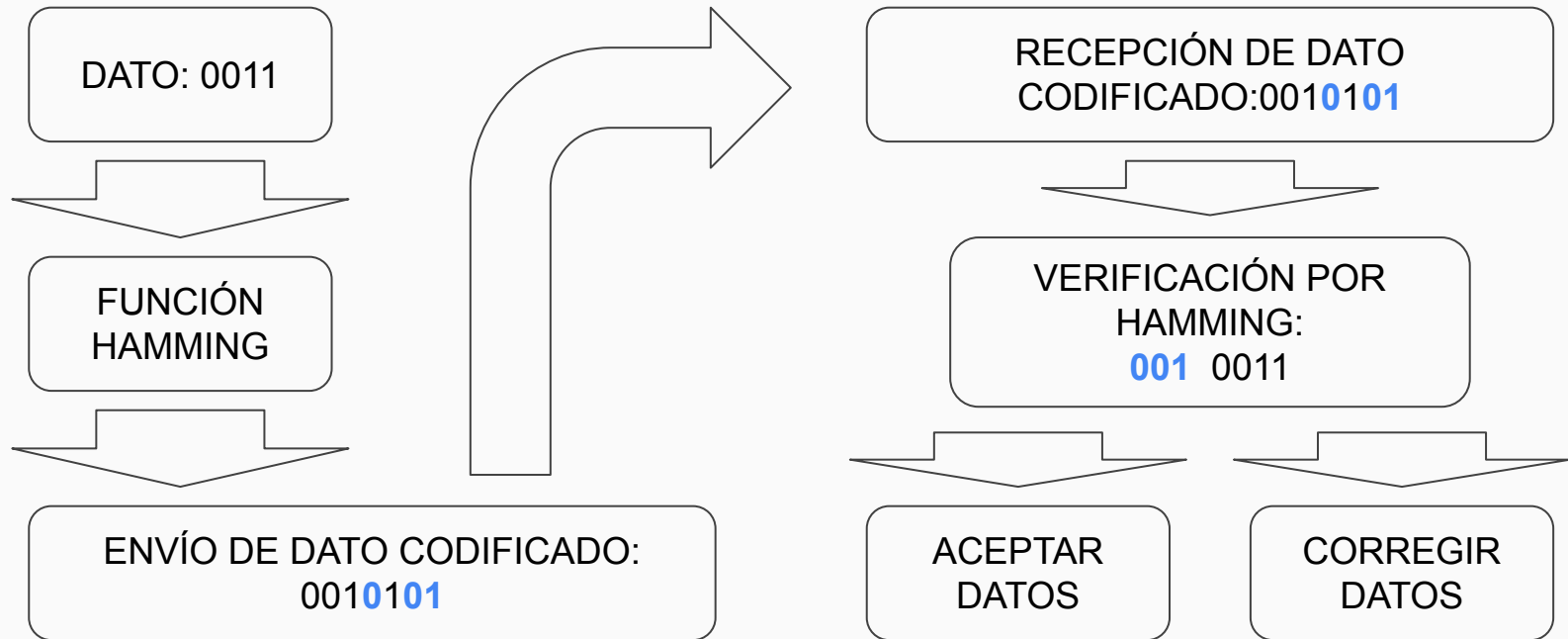
Redundancia: (El mensaje es dividido en dos partes)

- los bits de datos del mensaje
- los bits de redundancia para verificar el mensaje

Paridad:

- valor de los bits de redundancia
- bit paridad par: el bit tiene el valor de tal forma que el peso de la palabra sea par
- bit paridad impar: el bit tiene el valor de tal forma que el peso de la palabra sea impar

Diagrama de funcionamiento



Códigos de Hamming

- Código de corrección de errores fácil de implementar
- Idea: incluir más bits de redundancia y distribuirlos → bits erróneos distintos → resultados distintos y se pueden identificar bits erróneos
- Nomenclatura: (n° bits totales, n° bits info)
 - Ejemplo: $(8,7) \equiv$ 8 bits en total de los que 7 llevan información
- Código $(7,4)$ más típico ($d_H=3$)

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Códigos de Hamming

- Algoritmo:

- Posiciones potencia de 2 son bits de paridad ($2^0=1, 2, 4, 8, 16, \dots$)
- Resto son datos
- Cada bit de paridad calcula paridad de un conjunto de datos (no todos) determinados por la posición del bit de paridad. Bit 1 ($p_1:0001$) calcula paridad bits datos con LSB a 1.

Ejemplo: (11,7) p: bit de paridad datos= 0101001
d: bit de datos

[illegible]

Códigos de Hamming

- Algoritmo:

- Posiciones potencia de 2 son bits de paridad ($2^0=1, 2, 4, 8, 16...$)
- Resto son datos
- Cada bit de paridad calcula paridad de un conjunto de datos (no todos) determinados por la posición del bit de paridad. Bit 1 ($p_1:0001$) calcula paridad bits datos con LSB a 1.

Ejemplo: (11,7) p: bit de paridad datos= 0101001
d: bit de datos

	p_1	p_2	d_1	p_3	d_2	d_3	d_4	p_4	d_5	d_6	d_7
Posición	0001 (1)	0010 (2)	0011 (3)	0100 (4)	0101 (5)	0110 (6)	0111 (7)	1000 (8)	1001 (9)	1010 (10)	1011 (11)
Palabra original			0		1	0	1		0	0	1
p_1	1		0		1		1		0		1
p_2		0	0			0	1			0	1
p_3				0	1	0	1				
p_4								1	0	0	1
Palabra+paridad	1	0	0	0	1	0	1	1	0	0	1

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 10001011000

[illegible]

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 10001011000

[illegible]

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 10001011000

[illegible]

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 10001011000

[illegible]

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 10001011000

[illegible]

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 1000101100**0**

	p_1	p_2	d_1	p_3	d_2	d_3	d_4	p_4	d_5	d_6	d_7	Cálculo paridad	Paridad almacenada	Comprobación
Pos.	0001 (1)	0010 (2)	0011 (3)	0100 (4)	0101 (5)	0110 (6)	0111 (7)	1000 (8)	1001 (9)	1010 (10)	1011 (11)			
Pal.			0		1	0	1		0	0	0			
p_1	0		0		1		1		0		0			
p_2		1	0			0	1			0	0			
p_3				0	1	0	1							
p_4								0	0	0	0			

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 1000101100**0**

	p_1	p_2	d_1	p_3	d_2	d_3	d_4	p_4	d_5	d_6	d_7	Cálculo paridad	Paridad almacenada	Comprobación
Pos.	0001 (1)	0010 (2)	0011 (3)	0100 (4)	0101 (5)	0110 (6)	0111 (7)	1000 (8)	1001 (9)	1010 (10)	1011 (11)			
Pal.			0		1	0	1		0	0	0			
p_1	0		0		1		1		0		0	0	1	
p_2		1	0			0	1			0	0	1	0	
p_3				0	1	0	1					0	0	
p_4								0	0	0	0	0	1	

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 1000101100**0**

	p_1	p_2	d_1	p_3	d_2	d_3	d_4	p_4	d_5	d_6	d_7	Cálculo paridad	Paridad almacenada	Comprobación
Pos.	0001 (1)	0010 (2)	0011 (3)	0100 (4)	0101 (5)	0110 (6)	0111 (7)	1000 (8)	1001 (9)	1010 (10)	1011 (11)			
Pal.			0		1	0	1		0	0	0			
p_1	0		0		1		1		0		0	0	1	Error (1)
p_2		1	0			0	1			0	0	1	0	Error (1)
p_3				0	1	0	1					0	0	OK (0)
p_4								0	0	0	0	0	1	Error (1)

Códigos de Hamming

Palabra almacenada = 10001011001

Introducimos un error en un bit = 1000101100**0**

	p ₁	p ₂	d ₁	p ₃	d ₂	d ₃	d ₄	p ₄	d ₅	d ₆	d ₇	Cálculo paridad	Paridad almacenada	Comprobación
Pos.	0001 (1)	0010 (2)	0011 (3)	0100 (4)	0101 (5)	0110 (6)	0111 (7)	1000 (8)	1001 (9)	1010 (10)	1011 (11)			
Pal.			0		1	0	1		0	0	0			
p ₁	0		0		1		1		0		0	0	1	Error (1)
p ₂		1	0			0	1			0	0	1	0	Error (1)
p ₃				0	1	0	1					0	0	OK (0)
p ₄								0	0	0	0	0	1	Error (1)

p₄ p₃ p₂ p₁

datos

Comprobación paridad = 1 0 1 1 → 11 → Error bit 11 → 10001011001 → **0101001**